

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital
Carrera de Ingeniería en Software

Exposición con demostración práctica

Herramientas CASE

Aplicado al Proyecto Fin de Curso (PFC)

Caso de aplicación: SIMPA

Sistema Inteligente de Mantenimiento de Palma Africana
Subsistema modelado: Monitoreo Fitosanitario y Gestión de Labores Agrícolas

Asignatura:	Ingeniería de Requisitos
Docente:	Ing. Gleiston Guerrero Ulloa, PhD.
Paralelo:	4to Software A
Herramienta MDD utilizada:	Enterprise Architect (Sparx Systems)
Lenguaje objetivo:	C#

Integrantes del equipo y sus roles:

Integrante	Rol
Tigasi Sampedro Paul Alexander	Investigación del marco teórico, redacción completa del informe en LaTeX, documentación del modelo, elaboración del README del repositorio y capacitación al equipo en las funciones principales y el uso de Enterprise Architect.
Huilcapi León Denisses Fabiola	Configuración del generador de código en Enterprise Architect, ejecución del Forward Engineering y verificación de la generación y compilación del código resultante.
Vaca Romero David Octavio	Modelado UML del subsistema, implementación de la trazabilidad (Traceability), verificación de funcionalidades del sistema y grabación de la demo en video de la práctica.

Repositorio del proyecto:

[Link de repositorio de GitHub](#)

Resumen

El presente informe documenta la aplicación del ciclo completo de *Model-Driven Development* (MDD) sobre un subsistema representativo del Sistema Inteligente de Mantenimiento de Palma Africana (SIM-PA), correspondiente al Proyecto Fin de Curso del equipo. Se seleccionó Enterprise Architect como herramienta MDD por su soporte maduro de plantillas de generación de código en C#/.NET, alineado con la plataforma objetivo del PFC. Se modeló el subsistema de *Monitoreo Fitosanitario y Gestión de Labores Agrícolas*, compuesto por once clases del dominio con relaciones de asociación, composición, agregación y generalización. A partir de dicho modelo se configuró y ejecutó el mecanismo de generación de código, se verificó la compilación y ejecución de una unidad demostrable, y se evidenció el cierre del ciclo mediante una prueba de *roundtrip* controlada. El trabajo se organiza en un repositorio público de GitHub con evidencia de contribución de cada integrante.

Palabras clave: Model-Driven Development, MDA, UML, Enterprise Architect, generación de código, ingeniería de requisitos.

Índice

1. Marco teórico	3
1.1. Model-Driven Engineering (MDE), MDA y MDD	3
1.2. Transformaciones M2M y M2T	3
1.3. Metamodelo y perfiles UML	3
1.4. Forward, reverse y round-trip engineering	3
1.5. Estado del arte	4
2. Justificación de la selección de la herramienta	4
3. Descripción del subsistema del PFC modelado	4
4. Modelo UML de origen	5
4.1. Diagrama de clases	5
4.2. Clases del dominio y estereotipos	6
4.3. Relaciones del modelo	6
4.4. Trazabilidad	6
5. Configuración del generador	7
6. Proceso de generación y verificación del código	8
6.1. Correspondencia modelo → código	9
6.2. Limitaciones observadas del generador	10
7. Cierre del ciclo (roundtrip controlado)	10
8. Reparto del trabajo por integrante	13
9. Conclusiones	14
Anexos	15

1 Marco teórico

1.1 Model-Driven Engineering (MDE), MDA y MDD

Model-Driven Engineering (MDE) es el paradigma de desarrollo de software en el cual los modelos dejan de ser documentos de apoyo y pasan a constituir artefactos de primera clase del proceso: el modelo se convierte en la fuente autoritativa del sistema, y las implementaciones se derivan de él mediante transformaciones automatizadas [1], [2].

Model-Driven Architecture (MDA) es la iniciativa del *Object Management Group* (OMG) que estructura el desarrollo en tres niveles de abstracción [3]:

- **CIM** (*Computation Independent Model*): representa el dominio y las reglas de negocio, independiente de cualquier consideración computacional. En el caso del SIMPA corresponde a los procesos agrícolas del cultivo de palma africana.
- **PIM** (*Platform Independent Model*): describe la lógica del sistema sin atarse a una plataforma de despliegue concreta. Corresponde al diagrama de clases de dominio presentado en la Sección 4, previo a la aplicación del perfil C#/.NET.
- **PSM** (*Platform Specific Model*): el mismo modelo, enriquecido con estereotipos y valores etiquetados específicos de la plataforma objetivo (perfil «Entity» orientado a Entity Framework Core).

Model-Driven Development (MDD) es la aplicación práctica de MDE al ciclo de vida del software: los modelos guían la construcción del sistema y el código fuente se obtiene, total o parcialmente, mediante generadores automáticos [4], [5].

1.2 Transformaciones M2M y M2T

Las transformaciones **modelo-a-modelo** (M2M) producen un modelo destino a partir de uno de origen, ambos conformes a metamodelos (por ejemplo, PIM → PSM). El estándar OMG de referencia es *Query/View/Transformation* (QVT) v1.3 [6].

Las transformaciones **modelo-a-texto** (M2T) producen artefactos textuales —código fuente, configuración, documentación— a partir de un modelo. El estándar OMG de referencia es *MOF Model to Text* (MOFM2T) v1.0 [7]. En este trabajo, Enterprise Architect ejecuta una transformación M2T directa: del modelo de clases PSM se obtiene código fuente C# mediante sus plantillas de generación internas (*Code Templates*).

1.3 Metamodelo y perfiles UML

Un **metamodelo** define la sintaxis abstracta de un lenguaje de modelado; UML se define sobre MOF (*Meta Object Facility*) [8]. Los **perfiles UML** extienden el metamodelo mediante estereotipos, valores etiquetados y restricciones aplicables a un dominio o plataforma concreta. En el modelo del SIMPA se aplicó el estereotipo «Entity» sobre las clases de dominio, de modo que el generador de Enterprise Architect produzca clases persistentes mediante Entity Framework Core.

1.4 Forward, reverse y round-trip engineering

El *forward engineering* consiste en generar código a partir del modelo; el *reverse engineering*, en recuperar el modelo a partir del código; y el *round-trip engineering* mantiene ambas direcciones sincronizadas. Este trabajo demuestra el ciclo *forward* completo y, de forma controlada, un *roundtrip*: un cambio introducido en el modelo se refleja en el código tras la regeneración (Sección 7).

1.5 Estado del arte

El campo de las transformaciones de modelos cuenta con más de 60 herramientas catalogadas [9], y persisten retos abiertos en escalabilidad, interoperabilidad y adopción industrial [2], [10].

2 Justificación de la selección de la herramienta

El PFC del equipo (SIMPA) está desarrollado sobre la plataforma **C# / .NET**, con persistencia mediante **Entity Framework Core**. Frente a este requisito se evaluaron tres alternativas:

Herramienta	A favor	En contra (para este PFC)
Enterprise Architect	Plantillas de generación de código y documentación editables (<i>Code Templates</i>), MDG Technology para perfiles personalizados, trazabilidad nativa (<i>Traceability window, Relationship Matrix</i>).	Curva de aprendizaje inicial en la edición de plantillas.
StarUML	Generador oficial para C# vía extensión.	Extensiones de generación menos maduras para personalizar el mapeo de asociaciones a colecciones tipadas.
Visual Paradigm	<i>Instant Generator</i> robusto, plantillas ORM/JPA.	Foco principal en Java/JPA; el soporte de plantillas C#/EF Core es menos directo que en EA.

Se seleccionó **Enterprise Architect** por su capacidad de edición fina de plantillas de código y documentación (*Model Report*), su generación directa de código C# a partir de los diagramas, su soporte de trazabilidad integrada (empleada en la Sección 4) y por ser la herramienta con la que el equipo ya tenía experiencia previa, lo que facilitó el desarrollo de las prácticas sin contratiempos.

3 Descripción del subsistema del PFC modelado

El SIMPA es un sistema orientado a la gestión integral del cultivo de palma africana, cuyas funcionalidades principales incluyen: registro de plantaciones y lotes de cultivo; gestión de trabajadores y actividades agrícolas; registro de labores de riego, polinización y control fitosanitario; monitoreo de incidencias en el cultivo; detección de plagas y enfermedades mediante análisis de imágenes; generación de alertas y recomendaciones de tratamiento; predicción de producción y cosecha; y generación de reportes estadísticos e históricos.

Para esta actividad se seleccionó el subsistema de **Monitoreo Fitosanitario y Gestión de Labores Agrícolas**, por ser representativo del núcleo funcional del PFC y por integrar en un solo flujo el registro de labores agrícolas, la detección de plagas mediante análisis de imágenes y la generación de alertas con recomendaciones de tratamiento — cubriendo así una porción no trivial y demostrable del sistema completo.

Actores principales: agrónomo/supervisor de campo, trabajador agrícola, módulo de análisis de imágenes (servicio de IA del SIMPA).

Requisitos funcionales cubiertos por el subsistema modelado:

- RF-1: Registrar plantaciones y sus lotes de cultivo asociados.
- RF-2: Registrar labores agrícolas (riego, polinización, control fitosanitario) por lote y trabajador.
- RF-3: Registrar incidencias detectadas en un lote.
- RF-4: Asociar a una incidencia una detección de plaga obtenida por análisis de imágenes.
- RF-5: Generar alertas a partir de una detección de plaga.
- RF-6: Emitir recomendaciones de tratamiento asociadas a una alerta.

4 Modelo UML de origen

4.1 Diagrama de clases

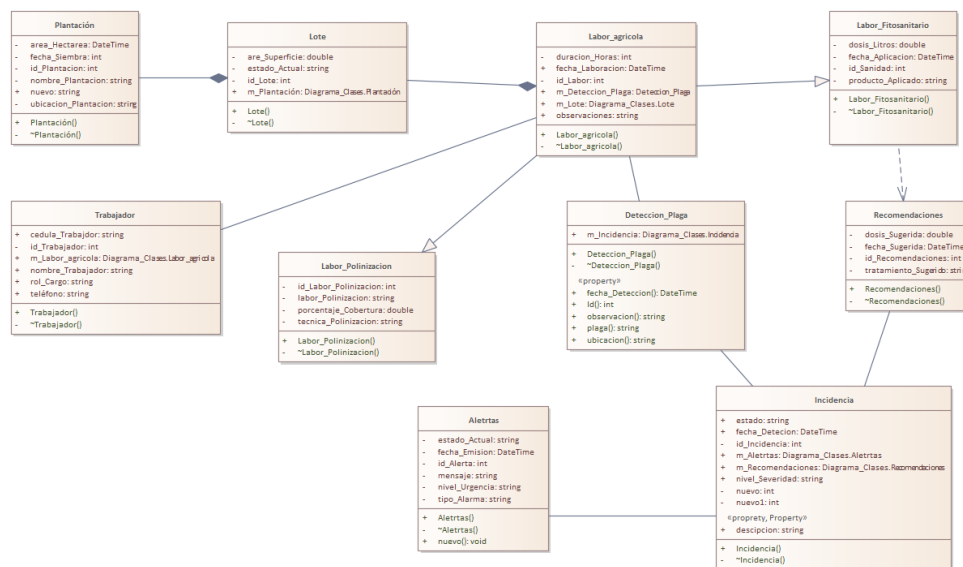


Figura 1: Diagrama de clases del subsistema de Monitoreo Fitosanitario y Gestión de Labores Agrícolas (SIMPA) finalizado y sincronizado con el código generado.

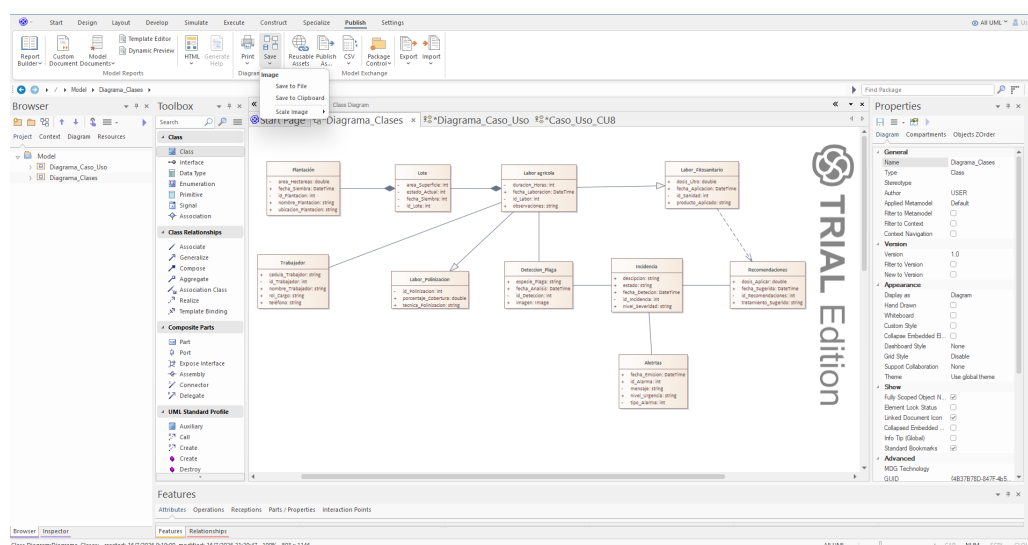


Figura 2: Captura de exportación del diagrama de clases del subsistema de Monitoreo Fitosanitario y Gestión de Labores Agrícolas (SIMPA) de Enterprise Architect.

4.2 Clases del dominio y estereotipos

Clase	Atributos principales	Estereotipo
Plantacion	id_Plantacion, nombre_Plantacion, ubicacion_Plantacion, area_Hectareas, fecha_Siembra	«Entity»
Lote	id_Lote, area_Superficie, estado_Actual, fecha_Siembra	«Entity»
Trabajador	id_Trabajador, cedula_Trabajador, nombre_Trabajador, rol_Cargo, telefono	«Entity»
LaborAgricola (base)	id_Labor, duracion_Horas, fecha_Laboracion, observaciones	«Entity»
LaborRiego	(verificar contra el modelo .qea)	«Entity»
LaborPolinizacion	id_Polinizacion, tecnica_Polinizacion, porcentaje_Cobertura	«Entity»
LaborFitosanitaria	producto_Aplicado, dosis_Litro, fecha_Aplicacion, id_Severidad	«Entity»
In idencia	id_Incidencia, fecha_Deteccion, descripcion, nivel_Severidad, estado	«Entity»
DeteccionPlaga	id_Deteccion, imagen (tipo Image), especie_Plaga, fecha_Analisis	«Entity»
Alertas	id_Alarma, tipo_Alarma, mensaje, nivel_Urgencia, fecha_Emision	«Entity»
Recomendaciones	id_Recomendaciones, tratamiento_Sugerido, dosis_Aplicar, fecha_Sugerida	«Entity»

4.3 Relaciones del modelo

- **Composición** Plantacion (1) — Lote (*): un lote no existe sin su plantación.
- **Asociación** Lote (1) — LaborAgricola (*): "programa".
- **Asociación** Trabajador (1) — LaborAgricola (*): ejecuta.
- **Generalización** LaborAgricola ← LaborRiego, LaborPolinizacion, LaborFitosanitaria.
- **Asociación** Lote (1) — Incidencia (*).
- **Agregación** Incidencia (1) — DeteccionPlaga (1).
- **Asociación** DeteccionPlaga (1) — Alerta (*).
- **Asociación** Alerta (1) — Recomendacion (0..1).

Diagramas complementarios: se recomienda incluir un diagrama de casos de uso (contexto de los tres actores) y un diagrama de estados para Incidencia (Detectada → En Tratamiento → Resuelta), en la medida en que aporten a la comprensión de la generación.

El archivo nativo del modelo (.qea) se encuentra en modelo/SIMPA_MDD.qea dentro del repositorio del equipo.

4.4 Trazabilidad

Se verificó, mediante la ventana *Traceability* de Enterprise Architect sobre la clase Incidencia, la consistencia de sus vínculos hacia DeteccionPlaga, Alerta y el caso de uso "Detectar plaga". Adicionalmente se empleó la *Relationship Matrix* (Lote × LaborAgricola) para confirmar que ninguna clase del modelo permanece desconectada.

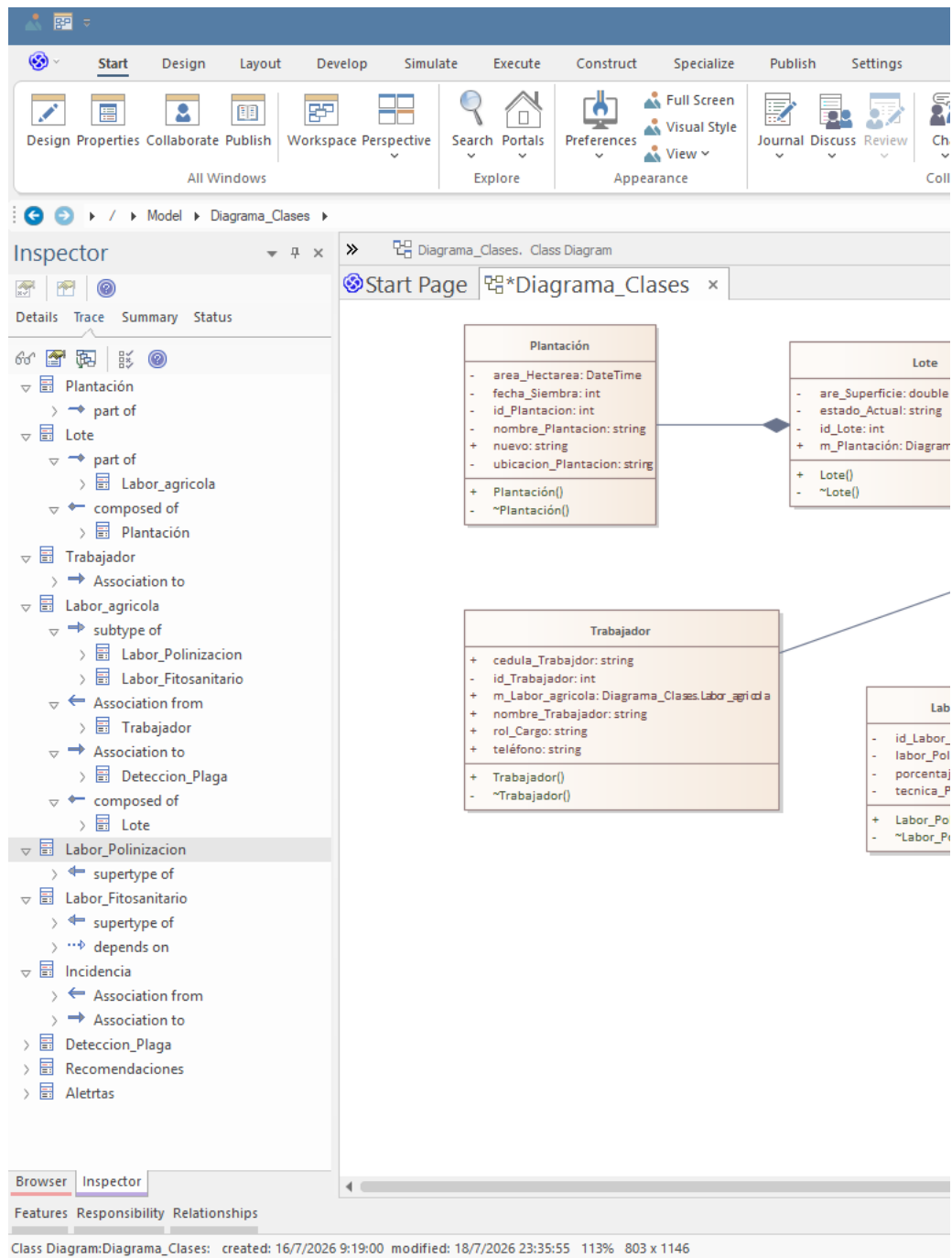


Figura 3: Captura de la sección de trazabilidad o "Trace" del diagrama de clase.

5 Configuración del generador

- **Generador utilizado:** plantilla nativa de Enterprise Architect para C#, personalizada mediante el editor de *Code Templates*.
- **Versión de Enterprise Architect:** 17.1.1716.
- **Lenguaje/framework objetivo:** C# 12 / .NET 8, Entity Framework Core 8.
- **Parámetros de generación:** directorio de salida `generado/Modelos/`; convención de nombres PascalCase para clases y propiedades; anotación `[Key]` sobre el atributo `id` de cada clase «Entity».

• Decisiones de mapeo documentadas:

- Asociación 1 a * → propiedad public ICollection<T>Nombre { get; set; }.
- Generalización UML → herencia de clases C# (class LaborRiego : LaborAgricola).
- Atributos tipados UML Date → DateTime de C#.

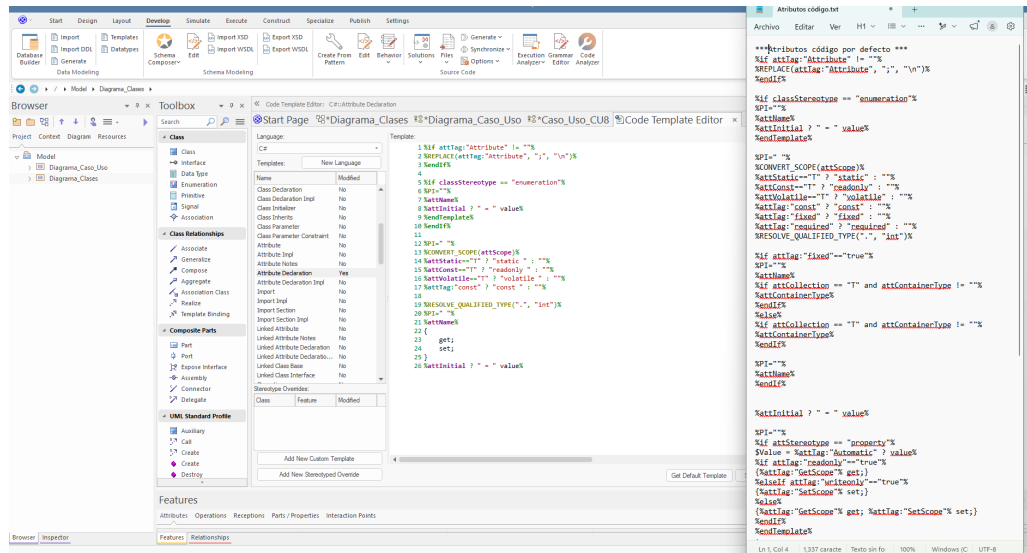


Figura 4: Captura de la personalización de la plantilla del CodeTemplateEditor para la generación del código del diagrama de clase.

6 Proceso de generación y verificación del código

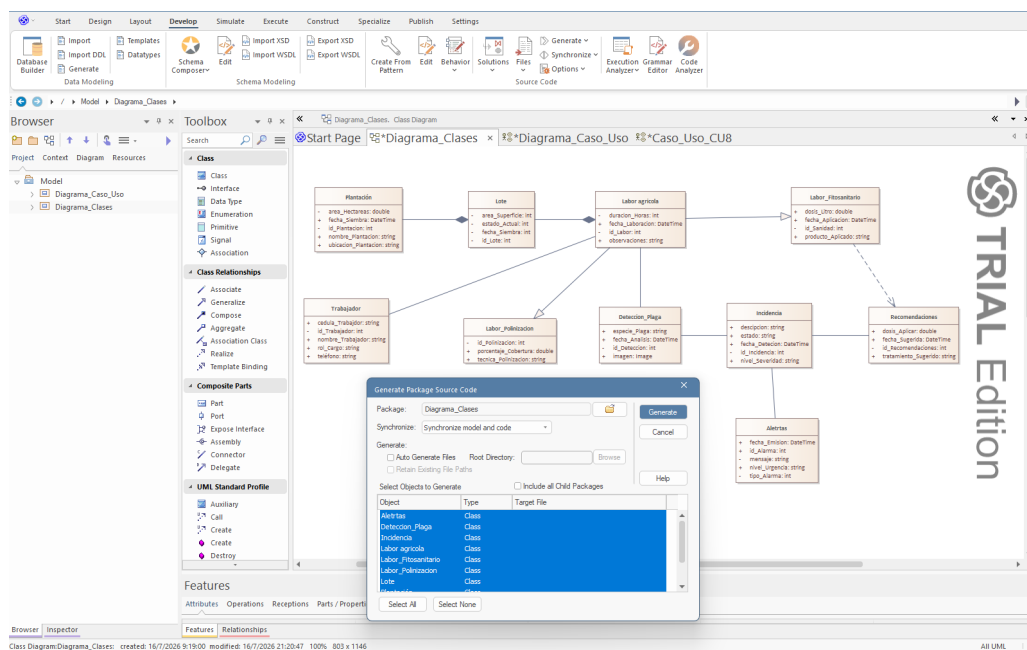


Figura 5: Captura de la invocación del generador en EA y el log de salida.

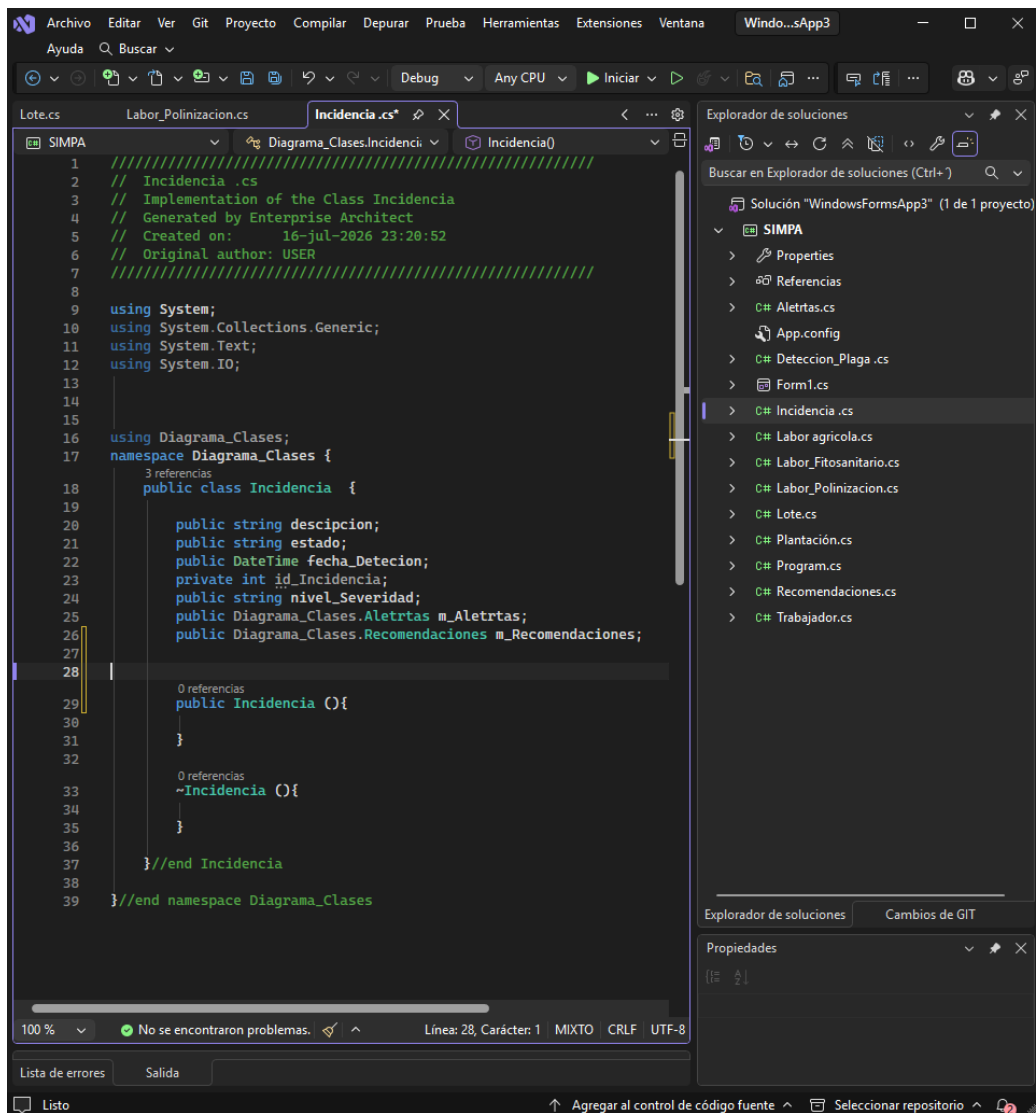


Figura 6: Captura del código C# generado en el explorador de soluciones de Visual Studio.

6.1 Correspondencia modelo → código

Elemento del modelo	Artefacto generado	Observación
Clase Incidencia	Incidencia.cs	Propiedades públicas + [Key] sobre id
Atributo nivel_Severidad	Campo public string Nivel_Severidad	Tipo UML String → string
Asociación Lote–LaborAgricola	public ICollection <LaborAgricola>Labores en Lote.cs	Mapeo 1 a *
Generalización LaborAgricola	class LaborRiego : LaborAgricola	Herencia simple de C#

6.2 Limitaciones observadas del generador

- El generador no produce las capas de *Service* ni *Controller*; estas se completan manualmente y se distinguen del código generado mediante su ubicación en carpetas separadas (generado/Servicios, generado/Controllers).
- Las validaciones de negocio (por ejemplo, rangos válidos de dosis_Litro) no se generan automáticamente y deben añadirse a mano sobre las clases parciales o en la capa de servicio.
- El generador no crea automáticamente las migraciones de Entity Framework Core; estas se generan aparte con `dotnet ef migrations add`.

7 Cierre del ciclo (roundtrip controlado)

Se ejecutó un cambio no trivial sobre el modelo: se agregó el atributo `coordenadasGPS` (tipo `String`) a la clase `Incidencia`, con el fin de registrar la ubicación exacta de la incidencia dentro del lote.

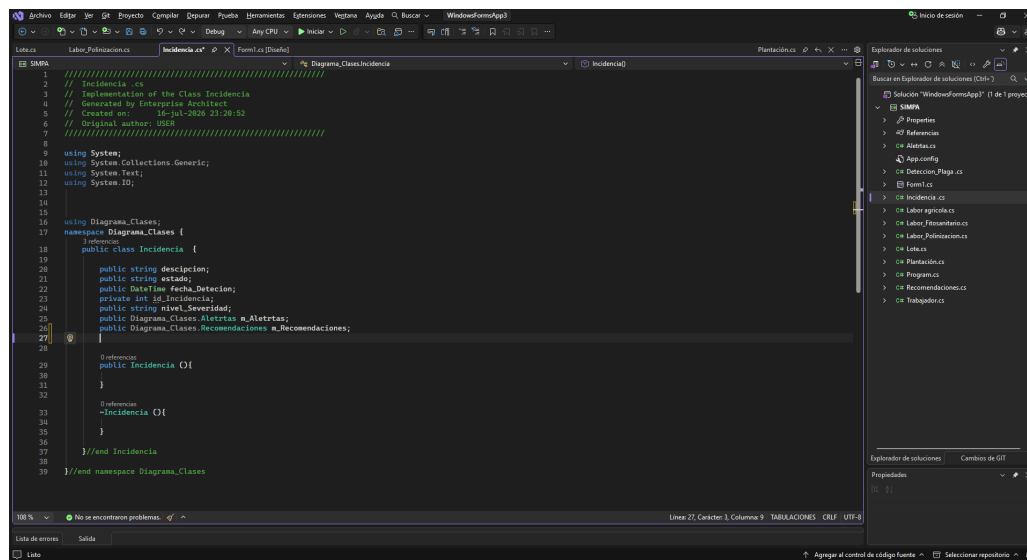


Figura 7: Modelo ANTES del cambio de plantilla y la agregación del atributo `coordenadasGPS` a la clase `Incidencia`.

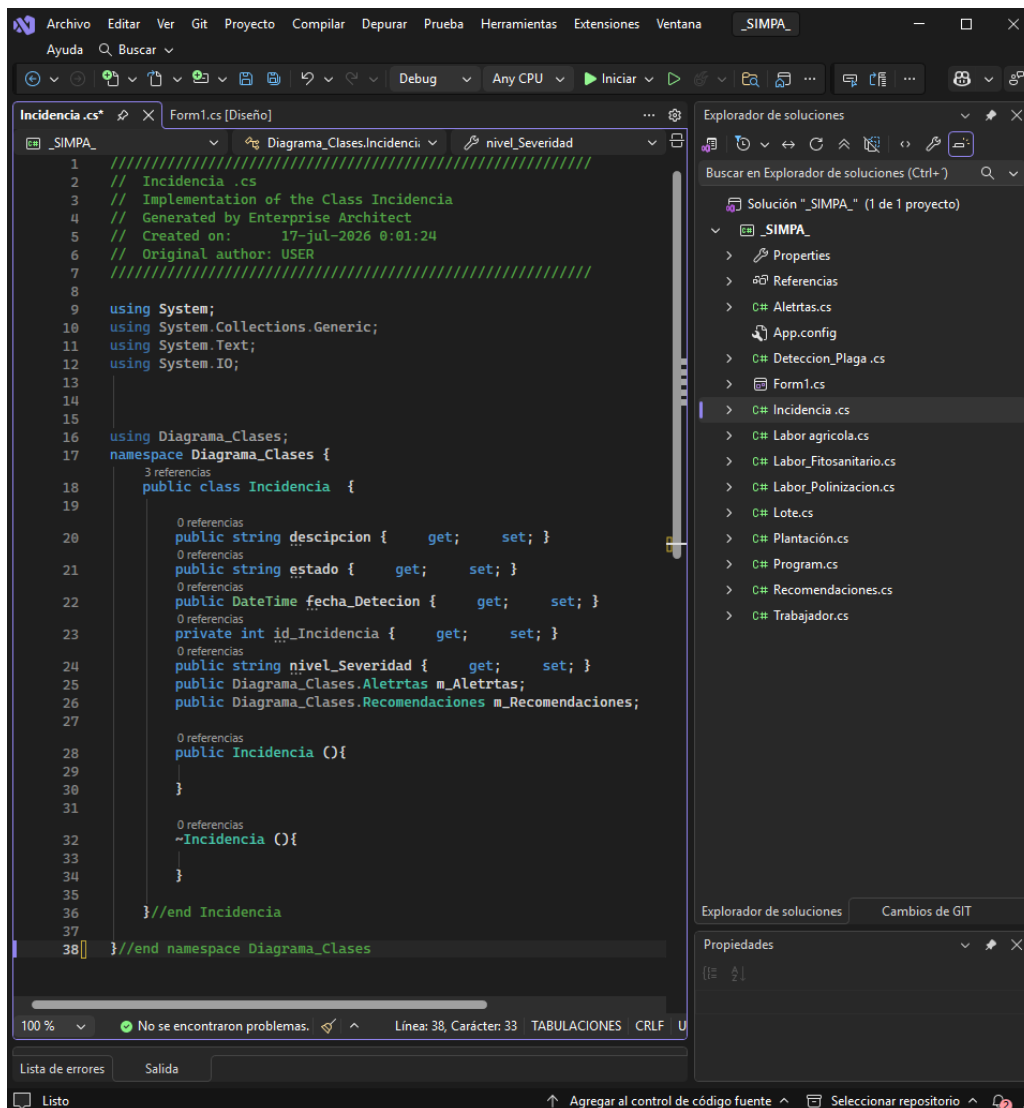


Figura 8: Modelo DESPUÉS del cambio — clase Incidencia con coordenadasGPS.

Tras regenerar, el archivo Incidencia.cs incorporó la nueva propiedad `public string CoordenadasGPS { get; set; }` sin afectar el código manual conservado en `generado/Servicios/`.

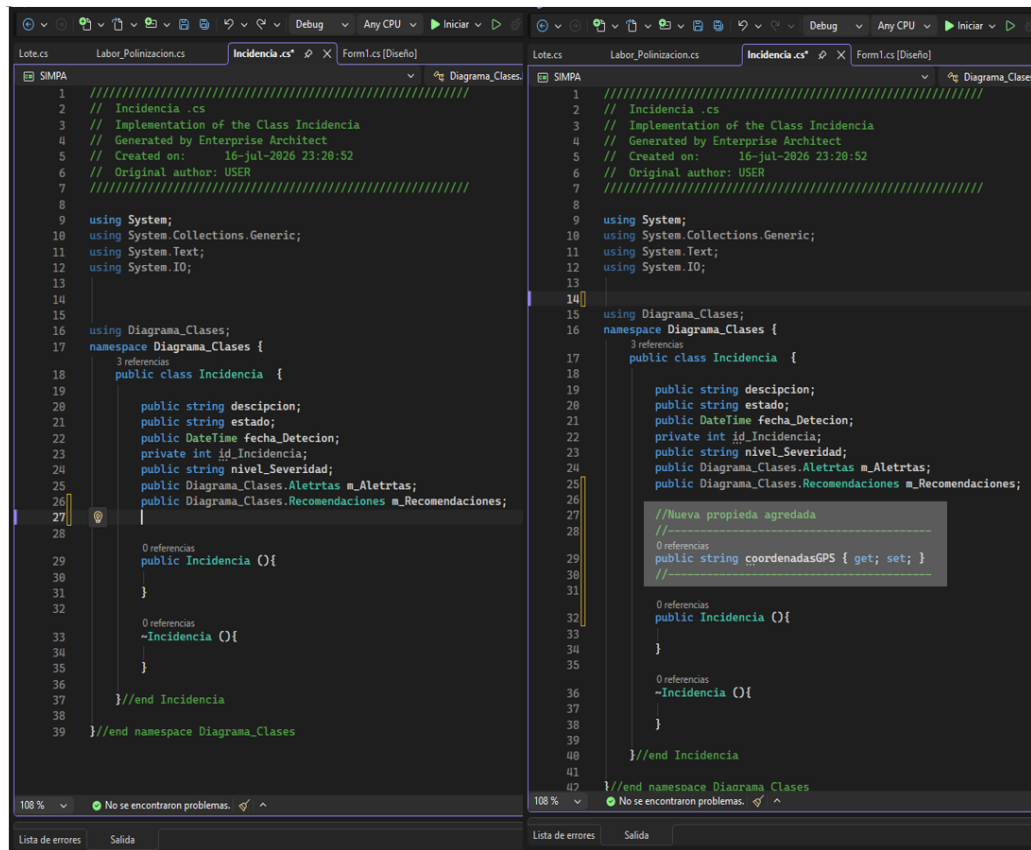


Figura 9: Código generado ANTES y DESPUÉS, con la propiedad CoordenadasGPS resaltada y la plantilla por defecto.

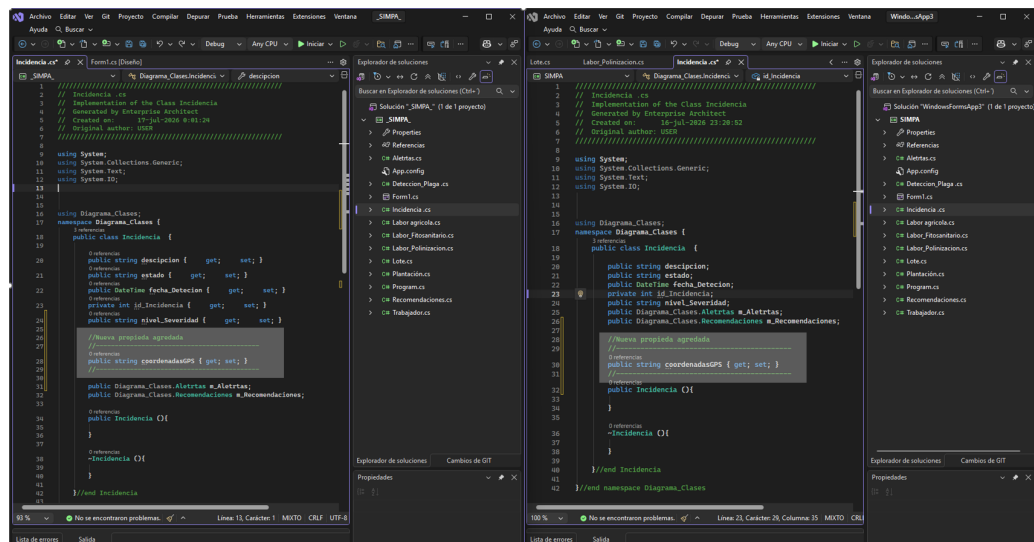


Figura 10: Código generado ANTES y DESPUÉS, con la propiedad CoordenadasGPS resaltada y la plantilla personalizada.

8 Reparto del trabajo por integrante

Integrante	Fases lideradas	Commits representativos (URL)
Tigasi Sampedro Paul Alexander	Documentación completa en Latex, creación y administración del repositorio GitHub, elaboración del README, investigación y exploración de Enterprise Architect, validación de funcionalidades CASE, apoyo al equipo en el uso de la herramienta e integración del informe final.	Ver commits
Huilcapi León Denisses Fabiola	Configuración del generador de código, ejecución del Forward Engineering, verificación de compilación, pruebas de Round-Trip Engineering, elaboración de diapositivas y organización de reuniones del equipo.	Ver commits
Vaca Romero David Octavio	Modelado UML, aplicación de estereotipos, implementación de la trazabilidad mediante Traceability, verificación de funcionalidades de Enterprise Architect y validación del modelo de Edit Code Template.	Ver commits

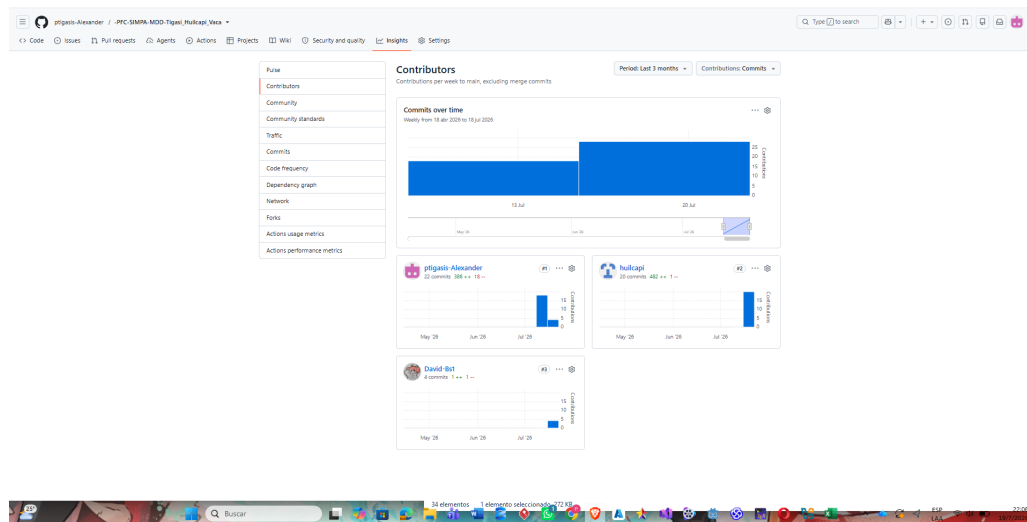


Figura 11: Captura de GitHub Insights → Contributors mostrando los commits diferenciados por integrante.

Todos los integrantes participan en la exposición operando en vivo la herramienta durante su segmento correspondiente, conforme al guion de exposición adjunto en [exposicion/](#).

9 Conclusiones

- La aplicación del ciclo MDD sobre el subsistema de Monitoreo Fitosanitario del SIMPA permitió reducir el trabajo manual de codificación de las clases de dominio, delegando en Enterprise Architect la generación de las propiedades, tipos y relaciones estructurales.

- La edición de las plantillas de código (*Code Templates*) resultó indispensable para que el mapeo de asociaciones y tipos fuera coherente con las convenciones de Entity Framework Core, en lugar de depender únicamente de la plantilla C# genérica.
- El generador no reemplaza el diseño de las capas de servicio y controladores, que continúan siendo responsabilidad del equipo; el valor de MDD radica en la generación confiable y trazable de la capa de datos, no en la automatización total del sistema.
- La prueba de *roundtrip* evidenció que el modelo puede evolucionar de forma segura sin perder el código manual, siempre que la política de regeneración aisle claramente las carpetas generadas de las escritas a mano.

Referencias

- [1] D. C. Schmidt, “Guest editor’s introduction: Model-driven engineering,” *Computer*, vol. 39, no. 2, pp. 25–31, 2006.
- [2] A. Bucchiarone, J. Cabot, R. F. Paige, and A. Pierantonio, “Grand challenges in model-driven engineering: An analysis of the state of the research,” *Software and Systems Modeling*, vol. 19, no. 1, pp. 5–13, 2020.
- [3] Object Management Group, “Model driven architecture (mda) guide,” OMG Document ormsc/14-06-01, 2014, revision 2.0.
- [4] B. Selic, “The pragmatics of model-driven development,” *IEEE Software*, vol. 20, no. 5, pp. 19–25, 2003.
- [5] M. Brambilla, J. Cabot, and M. Wimmer, *Model-Driven Software Engineering in Practice*, 2nd ed. San Rafael, CA: Morgan & Claypool, 2017.
- [6] Object Management Group, “Meta object facility (mof) 2.0 query/view/transformation specification,” OMG Document formal/16-06-03, 2016, version 1.3.
- [7] —, “Mof model to text transformation language (mofmt2t),” OMG Document formal/2008-01-16, 2008, version 1.0.
- [8] —, “Omg unified modeling language (omg uml),” OMG Document formal/17-12-05, 2017, version 2.5.1.
- [9] N. Kahani, M. Bagherzadeh, J. R. Cordy, J. Dingel, and D. Varró, “Survey and classification of model transformation tools,” *Software and Systems Modeling*, vol. 18, no. 4, pp. 2361–2397, 2019.
- [10] J. Whittle, J. Hutchinson, and M. Rouncefield, “The state of practice in model-driven engineering,” *IEEE Software*, vol. 31, no. 3, pp. 79–85, 2014.

Anexos

Anexo A. Ejemplo ilustrativo de clase generada

El siguiente fragmento ilustra la estructura de una clase generada por Enterprise Architect a partir del modelo (a completar/verificar con el código realmente generado por el equipo):

```
using System;
using System.Collections.Generic;
using System.Text;
using System.IO;

using Diagrama_Clases;
namespace Diagrama_Clases {
    public class Incidencia {

        public string descripcion {      get;      set; }
        public string estado {          get;      set; }
        public DateTime fecha_Deteccion {      get;      set; }
        private int id_Incidencia {          get;      set; }
        public string nivel_Severidad {      get;      set; }

        //Nueva propiedad agregada
        //-----
        public string coordenadasGPS { get; set; }
        //-----

        public Diagrama_Clases.Alertas m_Alertas;
        public Diagrama_Clases.Recomendaciones m_Recomendaciones;

        public Incidencia () {

        }

        ~Incidencia () {

        }

    } //end Incidencia

} //end namespace Diagrama_Clases
```


Anexo B. Estructura del repositorio

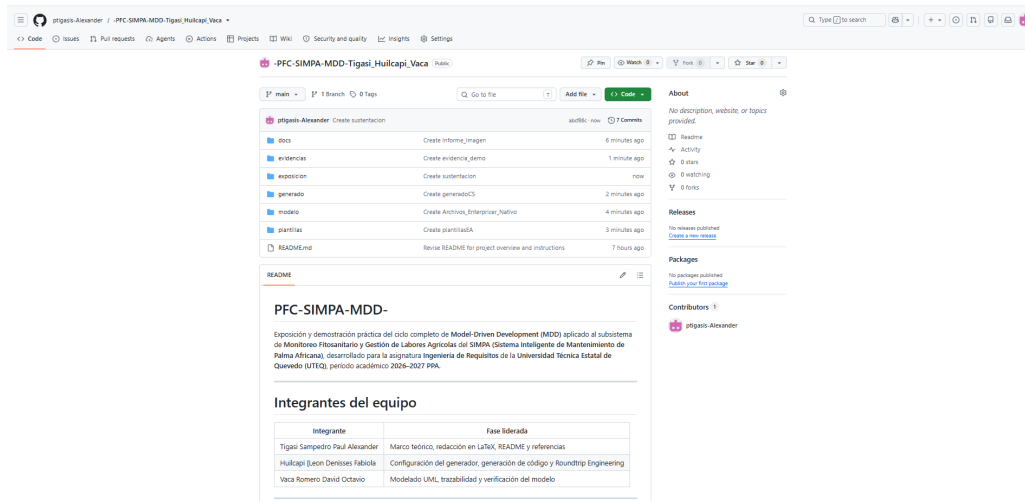


Figura 12: Captura de GitHub mostrando la estructura de carpetas del repositorio.

```
PFC-SIMPA-MDD-LaGeneracionIA/
README.md
docs/
informe.tex
informe.pdf
referencias.bib
figuras/
modelo/
SIMPA_MDD.qea
plantillas/
generado/
Modelos/
evidencias/
video-demo.mp4
exposicion/
diapositivas.pdf
```

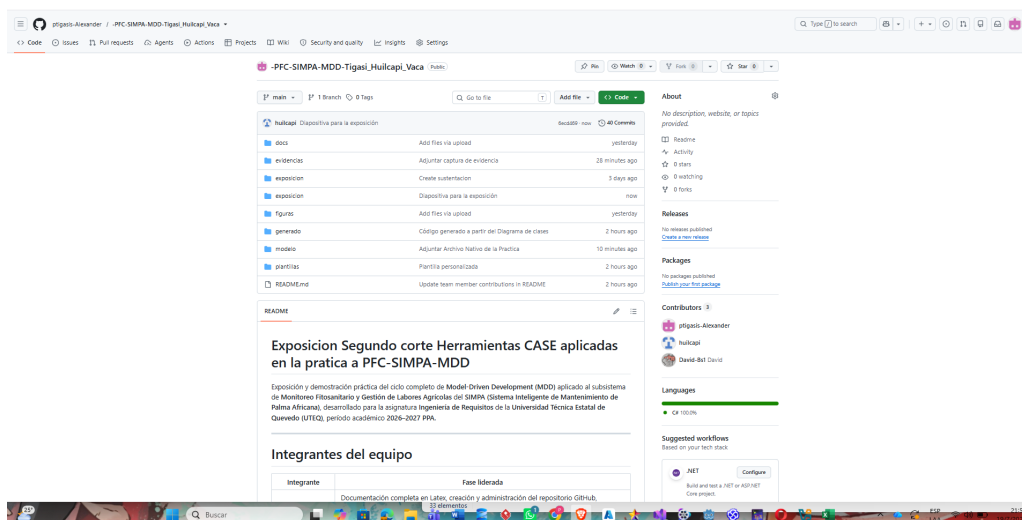


Figura 13: Captura de GitHub mostrando la union de los colaboradores del grupo.